INF201 - Deliverable3

Object-Oriented Game Simulator

Group members: Signe and Elli

Introduction

In this assigment, we have created a simple role-playing game simulator using object-oriented programming in Python. We have implemented classes for characters, dice and weapons, as well as functions to simulate battles. This notebook demonstrates and tests the different parts of program.

Task 1 - Character class

In this task, we implement the basic functionality of the Character class, which represents a character in the RPG simulator.

The class includes methods for attacking other characters, taking damage, and restoring health. The take_damage() and heal() methods ensure that HP stays within the valid range of 0 to hp_max.

We also implement the is_alive property, which returns whether the character has more than 0 HP remaining.

In [53]:
```
# Task 1 – Character class demonstration

from character import Character

# Create two characters with different attributes
hero = Character("Aria", 100, 15)
enemy = Character("Goblin", 60, 8)

# Display the characters' attributes
print("Hero:")
print("Name:", hero.name)
print("HP:", hero.hp)
print("Max HP:", hero.hp_max)
print("Damage:", hero.damage)
print("Weapon:", hero.weapon)

print("\nEnemy:")
print("Name:", enemy.name)
print("HP:", enemy.hp)
print("Max HP:", enemy.hp_max)
print("Damage:", enemy.damage)
print("Weapon:", enemy.weapon)
```

```
Hero:
Name: Aria
HP: 100
Max HP: 100
Damage: 15
Weapon: None

Enemy:
Name: Goblin
HP: 60
Max HP: 60
Damage: 8
Weapon: None
```

In [54]:
```python
# Demonstrate the attack method

print("Goblin's HP before attack:", enemy.hp)

hero.attack(enemy)

print("Goblin's HP after attack:", enemy.hp)
```

```
Goblin's HP before attack: 60
Goblin's HP after attack: 45
```

In [55]:
```python
# Demonstrate healing and HP limits

# Heal Goblin after the attack
enemy.heal(10)
print("HP after healing:", enemy.hp)

# Try healing beyond maximum HP
enemy.heal(100)
print("HP after excessive healing:", enemy.hp)

# Take more damage than remaining HP
enemy.take_damage(100)
print("HP after excessive damage:", enemy.hp)

# Check if Goblin is still alive
print("Is Goblin alive?", enemy.is_alive)
```

```
HP after healing: 55
HP after excessive healing: 60
HP after excessive damage: 0
Is Goblin alive? False
```

In [56]:
```python
# Demonstrate character introduction and string representations

hero.introduce()

print(str(hero))
print(repr(hero))
```

```
Hi, my name is Aria!
Character: Aria with 100 HP and 15 damage
<Character name=Aria hp=100 damage=15>
```

Results and Discussion

The tests demonstrate that the `Character` class works as expected.

When Aria attacked Goblin, Goblin's HP decreased from 60 to 45, confirming that the `attack()` and `take_damage()` methods work together correctly.

The healing tests showed that HP increased from 45 to 55, but could not exceed the maximum of 60. Similarly, excessive damage reduced HP to 0 without allowing negative values. This demonstrates how `min()` and `max()` enforce the HP limits.

The `is_alive` property returned `False` when Goblin reached 0 HP. The `introduce()`, `__str__()` and `__repr__()` methods also produced the expected character descriptions.

Overall, the tests confirm that the basic character functionality works correctly and provides a foundation for the specialised character classes and battle system.

Task 2 - Dice

In this task, we created a Dice class. The dice has six sides by default, but it is also possible to choose a diffrent number of sides.

The roll() method returns a random number between 1 and the number of sides.

In [57]:
```
from dice import Dice

d6 = Dice()
d8 = Dice(8)

print("Six-sided dice:", d6.roll())
print("Eight-sided dice:", d8.roll())
print("Calling the dice directly:", d6())

print("Number of sides (d6):", d6.sides)
print("Number of sides (d8):", d8.sides)

print("Dice representation:", repr(d6))
```

```
Six-sided dice: 5
Eight-sided dice: 2
Calling the dice directly: 5
Number of sides (d6): 6
Number of sides (d8): 8
Dice representation: Dice(sides=6)
```

Results and Discussion

The tests showed that both dice returned values within their expected ranges (1–6 and 1–8). The `__call__()` method also worked, allowing the dice to be rolled directly using `d6()`.

The `sides` attribute correctly displayed the number of sides for each dice, while `__repr__()` provided a readable representation of the object.

Overall, the results demonstrate that the Dice class works as expected in the tested examples.

Task 3 – Weapon Class

Overview and Task Description

In this task, we introduce the `Weapon` class, which uses a `Dice` object to calculate random damage through the `roll_damage()` method.

We also extend the `Character` class to support optional weapons. When a character has a weapon equipped, its attacks use the weapon's damage instead of the character's base damage.

In [58]:
```python
from dice import Dice
from items import Weapon

sword = Weapon("Sword", Dice(8))
axe = Weapon("Axe", Dice(12))

print("Weapon:", sword.name)
print("Damage dice:", sword.damage_dice)
print("Damage:", sword.roll_damage())

print("Weapon:", axe.name)
print("Damage dice:", axe.damage_dice)
print("Damage:", axe.roll_damage())
```

```
Weapon: Sword
Damage dice: Dice(sides=8)
Damage: 5
Weapon: Axe
Damage dice: Dice(sides=12)
Damage: 10
```

In [59]:
```python
# Task 3 – Character with and without a weapon

from character import Character

# Create characters
armed_hero = Character("Aria", damage=15, weapon=sword)
unarmed_hero = Character("Luna", damage=15)

# Create targets
target1 = Character("Goblin", hp_max=100)
target2 = Character("Orc", hp_max=100)

# Demonstrate attacks
armed_hero.attack(target1)
unarmed_hero.attack(target2)

print("Aria's weapon:", armed_hero.weapon.name)
print("Goblin HP after weapon attack:", target1.hp)

print("\nLuna's weapon:", unarmed_hero.weapon)
print("Orc HP after regular attack:", target2.hp)
```

```
Aria's weapon: Sword
Goblin HP after weapon attack: 97

Luna's weapon: None
Orc HP after regular attack: 85
```

Results and Discussion

The tests showed that the `Weapon` class successfully uses dice to generate random damage. The Sword (Dice(8)) dealt 6 damage, while the Axe (Dice(12)) dealt 8 damage.

When equipped with a weapon, the `Character` class used the weapon's dice roll instead of its base damage. Without a weapon, the character dealt its default damage of 15.

Overall, the results demonstrate how weapons and dice work together through object composition, allowing characters to deal variable damage.

Task 4 – Warrior and Mage

Overview and Task Description

In this task, we extend the `Character` class with two subclasses: `Warrior` and `Mage`.

The `Warrior` class introduces an `armor` attribute and overrides `take_damage()` to reduce incoming damage.

The `Mage` class introduces `mana` and `mana_max`, and overrides `attack()` to deal double damage using mana. When mana is insufficient, the mage performs a regular attack and restores 5 mana.

Both subclasses inherit the basic functionality of `Character`.

In [60]:
```python
# Task 4 – Warrior demonstration

from character import Warrior

# Create a warrior with 100 HP, 15 damage and 8 armor
warrior = Warrior("Thor", hp_max=100, damage=15, armor=8)

# Display attributes
print("Name:", warrior.name)
print("HP:", warrior.hp)
print("Max HP:", warrior.hp_max)
print("Damage:", warrior.damage)
print("Armor:", warrior.armor)
print("Weapon:", warrior.weapon)

# Demonstrate armor protection
warrior.take_damage(30)
print("\nHP after taking 30 damage:", warrior.hp)
```

```
# Demonstrate that armor can block all damage
warrior.take_damage(5)
print("HP after taking 5 additional damage:", warrior.hp)
```

```
Name: Thor
HP: 100
Max HP: 100
Damage: 15
Armor: 8
Weapon: None

HP after taking 30 damage: 78
HP after taking 5 additional damage: 78
```

In [61]:
```
# Task 4 – Mage demonstration

from character import Mage, Character

# Create a mage and an enemy
mage = Mage("Merlin", hp_max=100, damage=10)
target = Character("Goblin", hp_max=100)

# Display attributes
print("Name:", mage.name)
print("HP:", mage.hp)
print("Damage:", mage.damage)
print("Mana:", mage.mana)
print("Max mana:", mage.mana_max)
print("Weapon:", mage.weapon)

# Demonstrate magic attacks and mana consumption
for i in range(5):
    mage.attack(target)
    print(f"\nAttack {i + 1}:")
    print("Remaining mana:", mage.mana)
    print("Target HP:", target.hp)
```

```
Name: Merlin
HP: 100
Damage: 10
Mana: 30
Max mana: 30
Weapon: None

Attack 1:
Remaining mana: 20
Target HP: 80

Attack 2:
Remaining mana: 10
Target HP: 60

Attack 3:
Remaining mana: 0
Target HP: 40

Attack 4:
Remaining mana: 5
Target HP: 30

Attack 5:
Remaining mana: 10
Target HP: 20
```

In [62]:
```python
# Demonstrate Mage attacks with an equipped weapon

from dice import Dice
from items import Weapon

# Create a weapon and equip the mage
staff = Weapon("Magic Staff", Dice(6))

armed_mage = Mage("Gandalf", damage=10, mana=10, weapon=staff)
enemy_mage = Character("Orc", hp_max=100)

print("Weapon:", armed_mage.weapon.name)
print("Starting mana:", armed_mage.mana)

# First attack: magic (ignores weapon)
armed_mage.attack(enemy_mage)

print("\nAfter magic attack:")
print("Mana:", armed_mage.mana)
print("Enemy HP:", enemy_mage.hp)

# Second attack: regular attack (uses weapon)
armed_mage.attack(enemy_mage)

print("\nAfter regular attack:")
print("Mana:", armed_mage.mana)
print("Enemy HP:", enemy_mage.hp)
```

```
Weapon: Magic Staff
Starting mana: 10

After magic attack:
Mana: 0
Enemy HP: 80

After regular attack:
Mana: 5
Enemy HP: 78
```

Results and Discussion

The tests confirmed that both subclasses work as expected.

The Warrior's armor reduced incoming damage from 30 to 22, leaving 78 HP. An additional attack of 5 damage was completely blocked by the armor.

The Mage successfully used 10 mana to deal double damage (20 HP). When mana dropped below 10, regular attacks were performed and 5 mana was restored after each attack.

The weapon test also confirmed that magical attacks ignore equipped weapons, while regular attacks use weapon-based damage.

Overall, the results demonstrate how inheritance and method overriding allow different character specialisations.

Task 5 - Running Battles

Overview and Task Description

In this task, we created a `Battle` class to simulate combat between two characters.

The class has two attributes, `hero` and `enemy`, and two main methods:

- `play_round()` lets the hero attack first. If the enemy survives, it attacks back.
- `simulate_battle()` uses a while loop to repeat rounds until one character is defeated.

In [63]:
```python
# Task 5 – Battle demonstration

from character import Character
from battle import Battle

# Create two characters
hero = Character("Hero", hp_max=100, damage=20)
enemy = Character("Enemy", hp_max=50, damage=10)

# Create a battle
battle = Battle(hero, enemy)
```

```
# Play one round
battle.play_round()

# Display remaining HP
print("Hero HP:", hero.hp)
print("Enemy HP:", enemy.hp)
```

```
Hero HP: 90
Enemy HP: 30
```

In [64]:
```
# Task 5 - Simulate a complete battle

# Create two new characters
hero = Character("Hero", hp_max=100, damage=20)
enemy = Character("Enemy", hp_max=50, damage=10)

# Create a new battle
battle = Battle(hero, enemy)

# Simulate the battle until one character is defeated
battle.simulate_battle()

# Display the final results
print("Hero HP:", hero.hp)
print("Enemy HP:", enemy.hp)
print("Hero won:", hero.is_alive)
```

```
Hero HP: 80
Enemy HP: 0
Hero won: True
```

In [65]:
```
# Task 5 - Battle between Warrior and Mage

from character import Warrior, Mage

# Create two different characters: a Warrior and a Mage
hero = Warrior("Warrior", hp_max=100, damage=15, armor=5)
enemy = Mage("Mage", hp_max=80, damage=12, mana=30)

# Create and stimulate the battle
battle = Battle(hero, enemy)
battle.simulate_battle()

# Display the final results
print("Warrior HP:", hero.hp)
print("Mage HP:", enemy.hp)
print("Mage mana:", enemy.mana)
```

```
Warrior HP: 29
Mage HP: 0
Mage mana: 10
```

Results and Discussion

The tests confirmes that the Battle class works as expected. The `play_round()` method allowed the hero to attack first, followed by the enemy if it was still alive.

The `simulate_battle()` method repeated the rounds until one character was defeated.

The battle between Warrior and Mage also worked correctly, showing that the Battle class supports different character types.

Overall, the results demonstrate how the Battle class manages turn-based combat.

Use of Generative AI

We used ChatGPT to help us understand the tasks, explain Python concepts, and troubleshoot errors. We also used it for suggestions on code and testing.

We worked with the code ourselves, made changes, and tested the different functions to understand how they work.